Lab 04

Creating and Publishing an API Product through the Publisher

WSO2 API Manager 4.7.0 · Practical exercise

Before You Begin

URLs in this guide use placeholders. Replace them with the values of the API Manager you are using:

- **`<APIM_HOST>`** – host name or IP address of the API Manager server (localhost when API Manager runs on your own laptop).
- **`<HTTPS_PORT>`** – API Manager HTTPS port: 9443 plus any port offset (Publisher, Developer Portal, Admin Portal, Management Console, token endpoint).
- **`<GW_HTTPS_PORT>` / `<GW_HTTP_PORT>`** – API Gateway ports: 8243 (HTTPS) and 8280 (HTTP) plus any port offset.
- **`<APIM_HOME>`** – the folder where API Manager is extracted.

The servers use a self-signed certificate by default, so the browser shows a security warning the first time: choose Advanced and continue.

Training Objective

Learn how to create an API Product from resources of existing APIs, and publish it to the Developer Portal and the gateway using the Publisher.

Business Scenario

An API Product combines selected resources from one or more APIs into a single API that consumers can subscribe to. In this exercise, two APIs are created and published, and an API Product is then built from some of their resources.

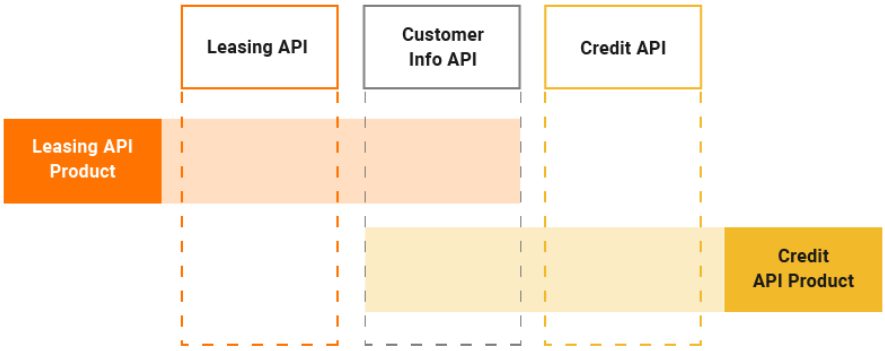
Let’s deviate a little from the PizzaShack application and look at a new business scenario.

A financial company ABC has 2 APIs as listed below:

- **Customer Info API** – used to access and update the financial information of the customers.
- **Leasing API** – used to access and update the information regarding the leases of the institution.

CC BY 4.0

Along with these 2 APIs, company ABC wants to combine a few of the existing resources from each API and expose a new API named **Customer Leasing API**, which can be used to achieve some of the functionalities of both the Leasing API and the Customer Info API. This can be achieved by creating an API Product.



High-Level Steps

- Start the ABC back ends and try them
- Add the Customer Info and Leasing APIs to the Publisher
- Create an API Product
- Invoke the API Product through the gateway

Part 1 – Start the ABC Back Ends and Try Them

Each API has its own back end service, as it would in a real bank: a customer service and a leasing service on different ports. A small program for company ABC that starts both mock back ends is provided with the lab files in the `abc-backend` folder. It keeps sample data in memory (customers 1001–1003 and leased assets 5001–5003). Start it first and check that both back ends answer before you create the APIs.

Back end	URL	Serves
Customer Info	`http://localhost:8091`	Customer Info API: /customers and /customers/{customerId}
Leasing	`http://localhost:8092`	Leasing API: /assets and /assets/{assetId}

1. Open a terminal (Command Prompt on Windows), go to the `abc-backend/bin` folder of the lab files and start the file for your operating system:

Operating system	Command
macOS (Apple silicon: M1, M2, M3, M4)	`./abc-backend-mac-arm64`
macOS (Intel)	`./abc-backend-mac-intel`
Windows	`abc-backend-windows.exe`
Linux	`./abc-backend-linux`

   The program starts both back ends and prints:

```
Customer Info back end listening on http://localhost:8091
Leasing back end listening on http://localhost:8092
```

CC BY 4.0

Note: leave this terminal open for the rest of the lab. Each back end prints one line for every request it receives, starting with its name (`[Customer Info]` or `[Leasing]`), so you can see which back end each call reaches. Stop the program with Ctrl+C when you finish.

Note: if macOS says the file cannot be opened because the developer cannot be verified, run `xattr -d com.apple.quarantine abc-backend-mac-*` in the same folder and start it again. On Windows, if SmartScreen appears, choose **More info** and then **Run anyway**. If port 8091 or 8092 is already in use, choose other ports, for example `./abc-backend-mac-arm64 -customer-port 9091 -leasing-port 9092`, and use those ports in the rest of the lab.

2. Open a second terminal and call the Customer Info back end directly:

```
curl http://localhost:8091/customers
curl http://localhost:8091/customers/1001
```

The first call lists three customers and the second returns one of them, for example:

```
{
  "customerId": "1001",
  "name": "Jane Wanjiku",
  "email": "jane.wanjiku@example.com",
  "phone": "+254700000001",
  "accountType": "SAVINGS",
  "balance": 152340.5,
  "currency": "KES"
}
```

3. Call the Leasing back end directly:

```
curl http://localhost:8092/assets
curl http://localhost:8092/assets/5001
```

The first call lists three leased assets and the second returns the lease for asset `5001` (a vehicle leased by customer 1001). You can also open these URLs in the browser.

4. Look at the first terminal: each back end has logged its own calls, for example `[Customer Info] GET /customers/1001` and `[Leasing] GET /assets/5001`. Each back end only knows its own resources: `http://localhost:8091/assets` returns 404. In the next part each API gets its own back end as its endpoint:

API	Endpoint
CustomerInformation	`http://localhost:8091`
Leasing	`http://localhost:8092`

Note: the back ends must run where API Manager can reach them. When API Manager runs on your laptop, localhost is correct. When you use a shared or remote API Manager, use the back end URLs given by your trainer in the rest of this lab.

Part 2 – Add the Individual APIs to the Publisher

Now that API Manager is set up and the users exist, create and publish the individual APIs.

5. Open the Publisher:

```
https://<APIM_HOST>:<HTTPS_PORT>/publisher
```

CC BY 4.0

Example:

```
https://localhost:9443/publisher
```

6. Log in as the user with the apicreator role you defined previously (**creator**).
7. Click **Create API**.
8. Under **REST API**, select **Import Open API**.
9. Select **OpenAPI File/Archive**, upload `customer-info-api.yaml` and click **Next**. The file is provided with the lab files.
10. Provide the API details as per the table below and click **Create**.

Field	Value	Description
Name	`CustomerInformation`	A unique name for identifying the API. On a shared server, add your initials (for example CustomerInformation-JD).
Display Name	`CustomerInformation`	Name of the API as you want it to appear in the Developer Portal.
Context	`/customer-info`	URI context path used by API consumers. It must be unique on the server (for example /customer-info-jd on a shared server).
Version	`1.0.0`	API version (in the form version.major.minor).
Endpoint	`http://localhost:8091`	The Customer Info back end started in Part 1. It is added as the production and sandbox endpoint.

11. Deploy the API: under **Deploy**, select **Deployments**, select the **Default** gateway and click **Deploy**.

    Note: deploy now, while logged in as creator. After an API is published, only a user with the publisher role can deploy new revisions.

12. Create another API: click **Create API → Import Open API → OpenAPI File/Archive**, upload `leasing-api.yaml` (provided with the lab files) and click **Next**.
13. Provide the API details as per the table below and click **Create**.

Field	Value	Description
Name	`Leasing`	A unique name for identifying the API (for example Leasing-JD on a shared server).
Display Name	`Leasing`	Name of the API as you want it to appear in the Developer Portal.
Context	`/leasing`	URI context path used by API consumers (for example /leasing-jd on a shared server).
Version	`1.0.0`	API version (in the form version.major.minor).
Endpoint	`http://localhost:8092`	The Leasing back end started in Part 1. It is added as the production and sandbox endpoint.

CC BY 4.0

14. Deploy the API in the same way (**Deploy → Deployments**, **Default** gateway, **Deploy**).

Part 3 – Create the API Product

15. Log out as **creator** and log in as **publisher**. Only the publisher can publish APIs and create API Products: the creator role cannot create API Products.

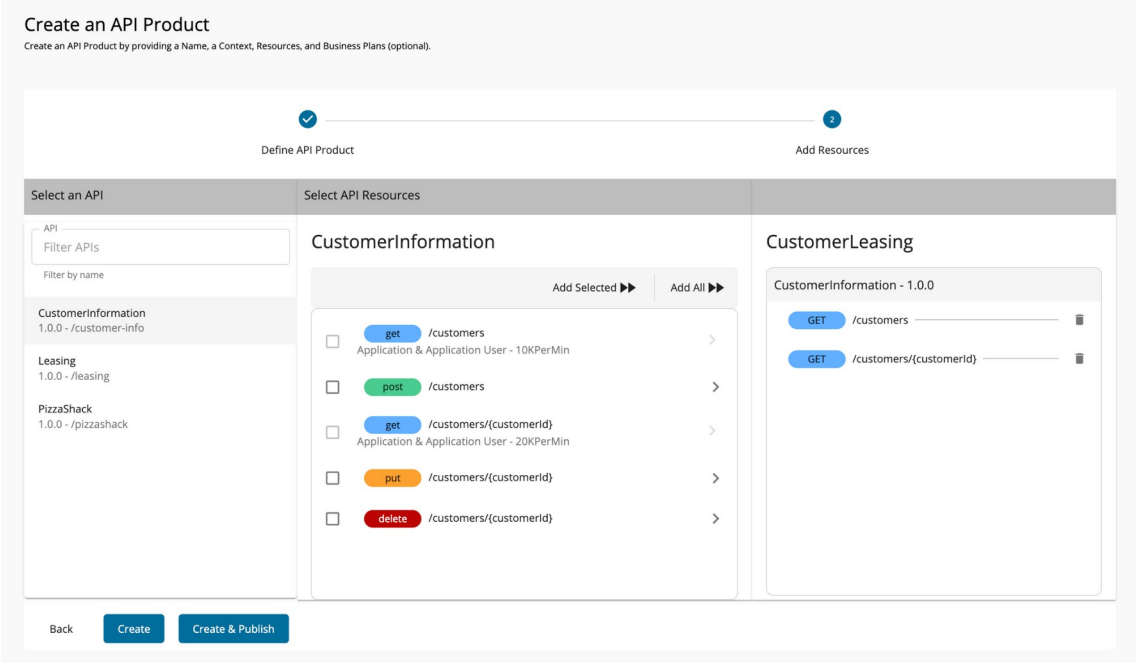
16. Publish both APIs: open each API, go to **Lifecycle** and click **Publish**.

17. Go to **API Products** and click **Create API Product**.

18. Enter the following details and click **Next**.

Field	Value	Description
Name	`CustomerLeasing`	A unique name for identifying the API Product.
Display Name	`CustomerLeasing`	Name of the API Product as you want it to appear in the Developer Portal.
Context	`/customer-leasing`	URI context path used by API consumers (for example /customer-leasing-jd on a shared server). Keep the version as 1.0.0.

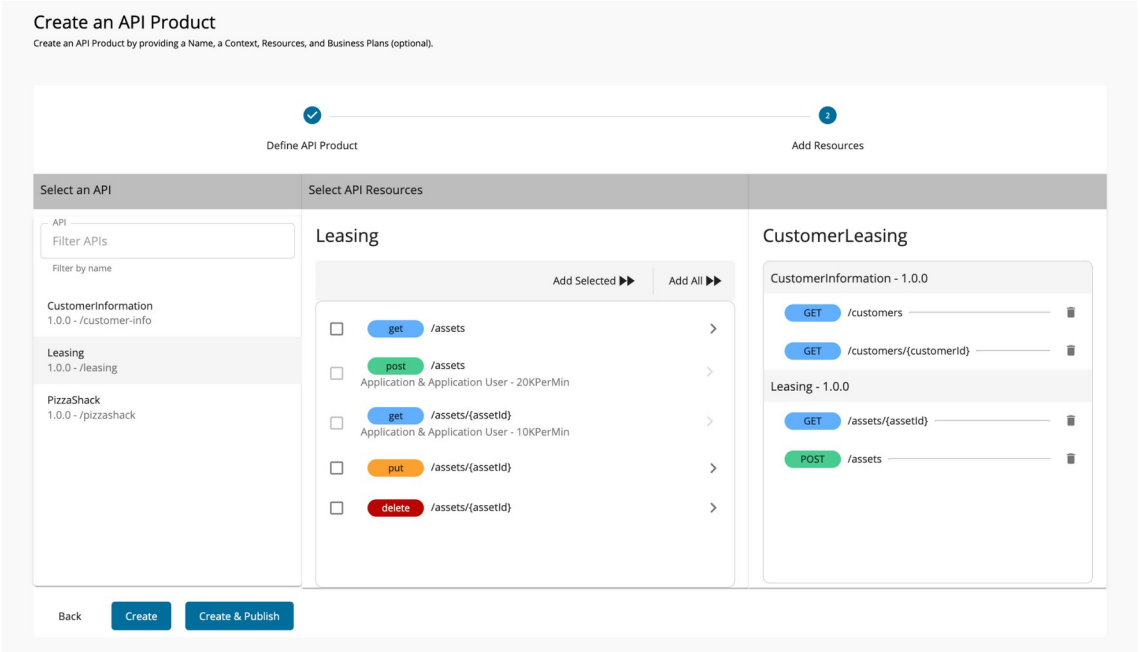
19. Select the following resources from the **CustomerInformation** API and click **Add Selected** (or click each resource to add it to the API Product), as shown below:
    - `GET /customers`
    - `GET /customers/{customerId}`



CC BY 4.0

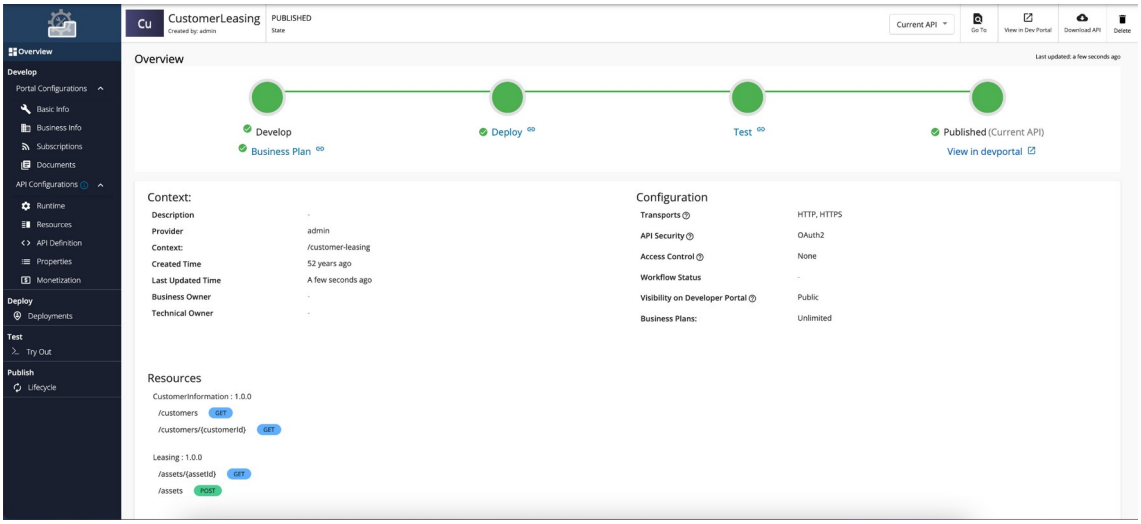
20. Select the following resources from the **Leasing** API and click **Add Selected**:

- `GET /assets/{assetId}`
- `POST /assets`



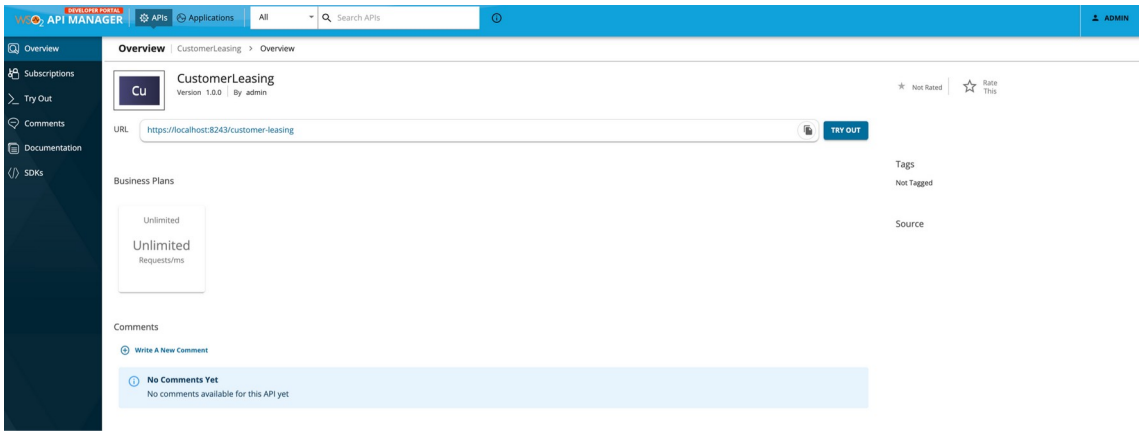
21. Click **Create & Publish**. The Publisher creates the API Product, creates a revision, deploys it to the gateway and publishes it in one step.

The API Product appears in the Publisher, as shown below:



CC BY 4.0

The API Product also appears in the Developer Portal, as shown below:



Part 4 – Invoke the API Product

The API Product is called on the gateway like any other API. The gateway checks the access token and sends each product resource to the back end of the API it came from: customer resources to the Customer Info back end (`8091`) and asset resources to the Leasing back end (`8092`). One product, two different back ends.

22. Subscribe an application to **CustomerLeasing** and generate an access token in the Developer Portal. Lab 05 shows how: you can come back to this part after Lab 05 and subscribe the application you create there to CustomerLeasing as well.

23. Call the product resources through the gateway, replacing `<ACCESS_TOKEN>` with your token:

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>"
https://<APIM_HOST>:<GW_HTTPS_PORT>/customer-leasing/1.0.0/customers
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>"
https://<APIM_HOST>:<GW_HTTPS_PORT>/customer-leasing/1.0.0/customers/1001
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>"
https://<APIM_HOST>:<GW_HTTPS_PORT>/customer-leasing/1.0.0/assets/5001
```

Example:

```
curl -k -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ"
https://localhost:8243/customer-leasing/1.0.0/customers
curl -k -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ"
https://localhost:8243/customer-leasing/1.0.0/customers/1001
curl -k -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ"
https://localhost:8243/customer-leasing/1.0.0/assets/5001
```

The responses are the same as the direct calls in Part 1. The back end terminal shows that one product reached two back ends: `[Customer Info] GET /customers`, `[Customer Info] GET /customers/1001` and `[Leasing] GET /assets/5001`. The gateway removes the product context `/customer-leasing/1.0.0` and sends the resource path to the back end of the API that owns the resource.

Tip: if the first call right after subscribing returns HTTP 500, the gateway has not finished loading the new subscription yet. Wait a few seconds and run the call again.

CC BY 4.0

24. Create a lease through the product (`POST /assets`):

macOS / Linux

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" -H "Content-Type:
application/json" -d '{"customerId":"1002","type":"VEHICLE","description":"Mazda
CX-5","leaseAmount":2500000,"monthlyRental":75000,"termMonths":36}'
https://<APIM_HOST>:<GW_HTTPS_PORT>/customer-leasing/1.0.0/assets
```

Example – macOS / Linux:

```
curl -k -X POST -H "Authorization: Bearer
eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" -H "Content-Type: application/json"
-d '{"customerId":"1002","type":"VEHICLE","description":"Mazda
CX-5","leaseAmount":2500000,"monthlyRental":75000,"termMonths":36}'
https://localhost:8243/customer-leasing/1.0.0/assets
```

Windows

```
curl -k -X POST -H "Authorization: Bearer <ACCESS_TOKEN>" -H "Content-Type:
application/json" -d "{\"customerId\":\"1002\",\"type\":\"VEHICLE\",\"description\":
\"Mazda CX-5\",\"leaseAmount\":2500000,\"monthlyRental\":75000,\"termMonths\":36}"
https://<APIM_HOST>:<GW_HTTPS_PORT>/customer-leasing/1.0.0/assets
```

Example – Windows:

```
curl -k -X POST -H "Authorization: Bearer
eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ" -H "Content-Type: application/json"
-d "{\"customerId\":\"1002\",\"type\":\"VEHICLE\",\"description\":\"Mazda CX-5\",
\"leaseAmount\":2500000,\"monthlyRental\":75000,\"termMonths\":36}"
https://localhost:8243/customer-leasing/1.0.0/assets
```

The gateway returns `201` and the new lease from the Leasing back end, with a new asset ID (for example `5004`) and status `PENDING`.

25. Call a resource that is not part of the product, for example `GET /assets`:

```
curl -k -H "Authorization: Bearer <ACCESS_TOKEN>"
https://<APIM_HOST>:<GW_HTTPS_PORT>/customer-leasing/1.0.0/assets
```

Example:

```
curl -k -H "Authorization: Bearer eyJ4NXQiOiJNell4.eyJzdWIiOiJhZG1pbiJ9.ZXhhbXBsZQ"
https://localhost:8243/customer-leasing/1.0.0/assets
```

The gateway rejects the call with HTTP 405 and nothing new appears in the back end terminal. Only the resources selected in the product can be called through it, even though the Leasing API itself has GET /assets:

```
{"code":"405","type":"Status report","message":"Runtime Error","description":"Method
not allowed for given API resource"}
```

Expected Outcome

The CustomerInformation and Leasing APIs each point to their own back end, and the CustomerLeasing API Product combines resources from both, is published in the Developer Portal, and reaches both back ends through the gateway.

CC BY 4.0